

Lecture 13

Normalization for Relational Database

Objectives

- First Normal Form
- Second Normal Form
- Third Normal Form
- Boyce-Codd Normal Form
- De-normalized

- Reference: Chapter 14 & 15

Data normalization

- Normalization is a formal process for deciding which attributes should be grouped together in a relation
 - Objective: to validate and improve a logical design so that it satisfies certain constraints that avoid unnecessary duplication of data
 - Definition: the process of decomposing relations with anomalies to produce smaller, well-structured relations

First Normal Form

- Disallows
 - composite attributes
 - multivalued attributes
 - **nested relations**; attributes whose values for an *individual tuple* are non-atomic
- Considered to be part of the definition of relation

Example: Normalization into 1NF

(a)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
-------	----------------	----------	------------



(b)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Example: Normalization nested relations into 1NF

(a)

EMP_PROJ

		Projs	
Ssn	Ename	Pnumber	Hours

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, AliciaJ.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

EMP_PROJ1

<u>Ssn</u>	Ename
------------	-------

EMP_PROJ2

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------

Second Normal Form

- Uses the concepts of **FDs**, **primary key**
- Definitions
 - **Prime attribute**: An attribute that is member of the primary key K
 - **Full functional dependency**: a FD $Y \rightarrow Z$ where removal of any attribute from Y means the FD does not hold any more

EMP_PROJ

Ssn	Ename	Pnumber	Hours
-----	-------	---------	-------

- **Examples:**
 - $\{SSN, PNumber\} \rightarrow Hours$ is a full FD since neither $SSN \rightarrow Hours$ nor $PNumber \rightarrow Hours$ hold
 - $\{SSN, PNumber\} \rightarrow EName$ is not a full FD (it is called a partial dependency) since $SSN \rightarrow EName$ also holds

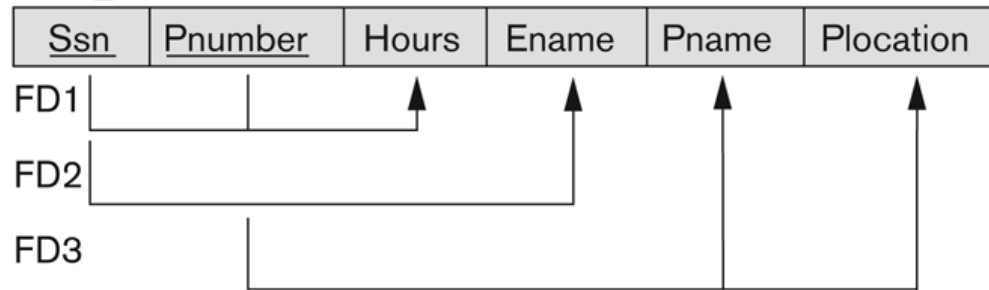
Second Normal Form (2)

- A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on the primary key
- R can be decomposed into 2NF relations via the process of 2NF normalization

Example Normalizing into 2NF

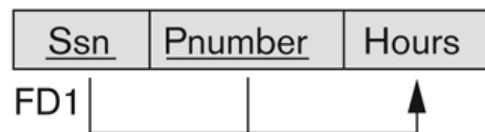
(a)

EMP_PROJ

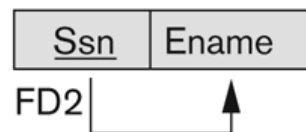


2NF Normalization

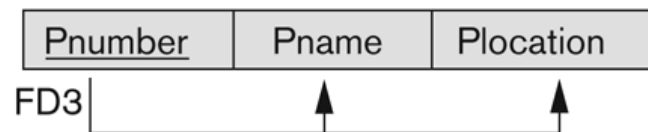
EP1



EP2



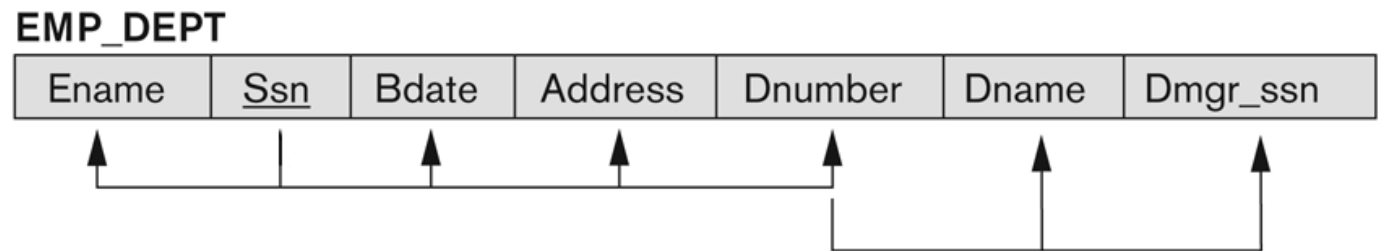
EP3



Third Normal Form

- Definition:
 - **Transitive functional dependency:**
FD $X \rightarrow Z$ that can be derived from two FDs
 $X \rightarrow Y$ and $Y \rightarrow Z$

- Examples:



Third Normal Form (2)

- Definition: A relation schema R is in **third normal form (3NF)** if
 - it is in 2NF *and*
 - no non-prime attribute A in R is **transitively dependent** on the primary key
- R can be decomposed into 3NF relations via the process of 3NF normalization
- **Note:**
 - In $X \rightarrow Y$ and $Y \rightarrow Z$, with X as the primary key, we consider this a problem only if Y is not a candidate key.
 - When Y is a candidate key, there is no problem with the transitive dependency .
 - E.g., Consider EMP (SSN, Emp#, Salary).
 - Here, $SSN \rightarrow Emp\#$, $Emp\# \rightarrow Salary$ and Emp# is a candidate key.

Third Normal Form (3)

- Alternative Definition: A relation schema R is in **third normal form (3NF)** if every non-prime attribute of R meets both of the following conditions:
 - It is fully functional dependent on every key of R
 - It is nontransitively dependent on every key of R

Normal Forms Defined Informally

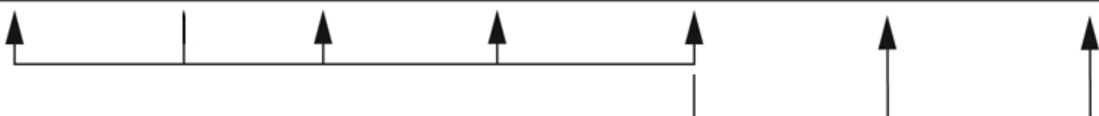
- 1st normal form
 - if and only if all columns contain only atomic values
- 2nd normal form
 - if and only if it is in 1NF and
 - Every nonkey attribute is fully dependent on the primary key
- 3rd normal form
 - must be in 2NF and no transitive dependencies may exist within the relation.
 - A *transitive dependency* is when an attribute is indirectly functionally dependent on the key (that is, the dependency is through another nonkey attribute).

Example Normalizing into 3NF

(b)

EMP_DEPT

Ename	<u>Ssn</u>	Bdate	Address	Dnumber	Dname	Dmgr_ssn
-------	------------	-------	---------	---------	-------	----------



3NF Normalization

ED1

Ename	<u>Ssn</u>	Bdate	Address	Dnumber
-------	------------	-------	---------	---------



ED2

<u>Dnumber</u>	Dname	Dmgr_ssn
----------------	-------	----------



General Normal Form Definitions (For Multiple Keys)

- The above definitions consider the primary key only
- The following more general definitions take into account relations with multiple candidate keys
- A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on *every* key of R

General Normal Form Definitions (2)

- Definition:
 - **Superkey** of relation schema R - a set of attributes S of R that contains a key of R
 - A relation schema R is in **third normal form (3NF)** if whenever a FD $X \rightarrow A$ holds in R, then either:
 - (a) X is a superkey of R, or
 - (b) A is a prime attribute of R
- **NOTE:** Boyce-Codd normal form disallows condition (b) above

BCNF (Boyce-Codd Normal Form)

- A relation schema R is in **Boyce-Codd Normal Form (BCNF)** if whenever an **FD** $X \rightarrow A$ holds in R , then X is a **superkey** of R
- Each normal form is strictly stronger than the previous one
 - Every 2NF relation is in 1NF
 - Every 3NF relation is in 2NF
 - Every BCNF relation is in 3NF
- There exist relations that are in 3NF but not in BCNF
- The goal is to have each relation in BCNF (or 3NF)

illustration: Boyce-Codd normal form

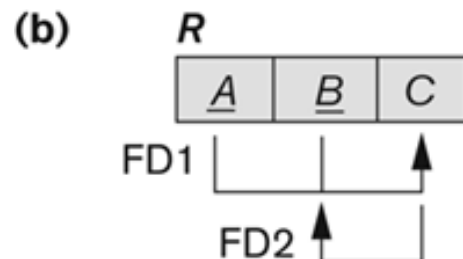
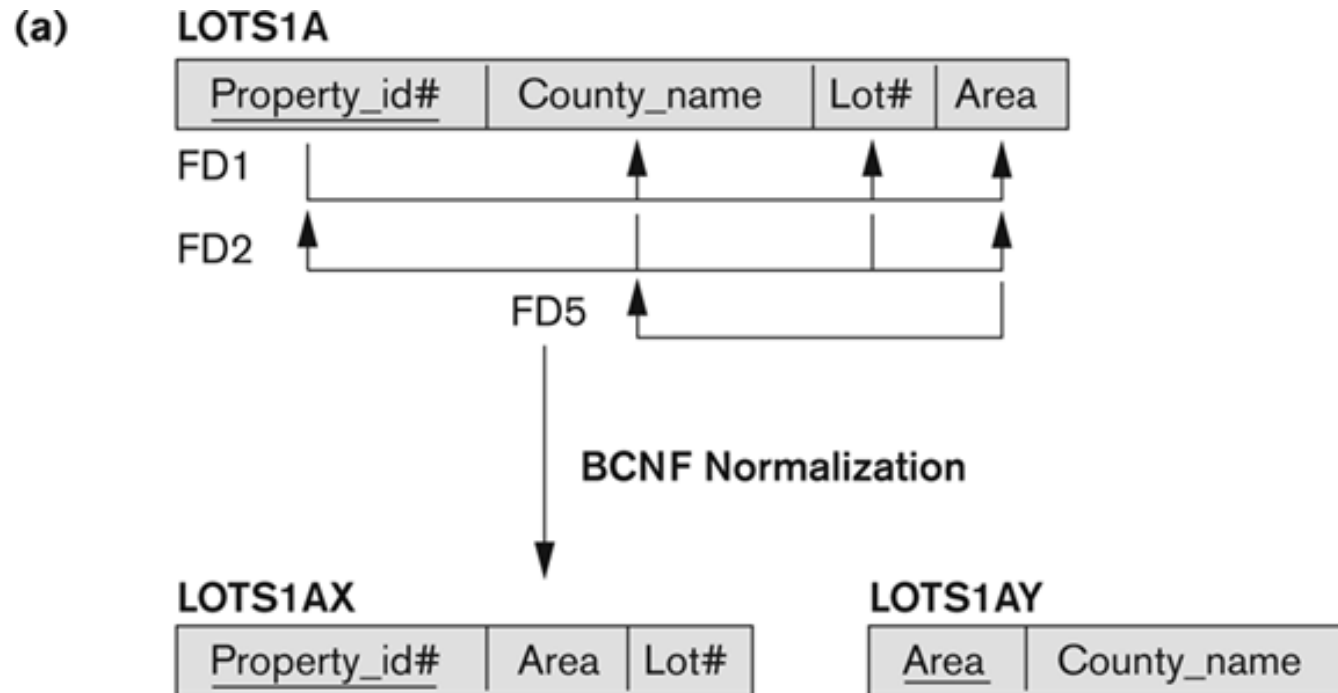


illustration: a relation **TEACH** that is in 3NF but not in BCNF

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

- Based on functional dependencies that takes into account all candidate keys in a relation.
- For a relation with only one candidate key, 3NF and BCNF are equivalent.
- A relation is in BCNF, if and only if every determinant is a candidate key.
- Violation of BCNF may occur in a relation that
 - contains 2 (or more) composite keys
 - which overlap and share at least 1 attribute

3NF to BCNF

- Identify all candidate keys in the relation.
- Identify all functional dependencies in the relation.
- If functional dependencies exists in the relation where their determinants are not candidate keys for the relation, remove the functional dependencies by placing them in a new relation along with a copy of their determinant.

De-normalized Designs

- When a normalized design is unnatural, awkward, or results in unacceptable performance, a de-normalized design is preferred
- Materialized views, which may implement the following:
 - Storing the count of the "many" objects in a one-to-many relationship as an attribute of the "one" relation
 - Adding attributes to a relation from another relation with which it will be joined

De-normalized Designs

- Example
 - Normalized relation
 - CUSTOMER (CustNumber, CustName, Zip)
 - CODES (Zip, City, State)
 - De-Normalized relations
 - CUSTOMER (CustNumber, CustName, City, State, Zip)

- Give the relation schema:

Projects(ProjectId, ProjectName, CustomerId)

ProjectDetails(ProjectId, ItemId, ActivityId, EmployeeId, WorkDate, TimeSpent)

Customers(CustomerId, CustomerName)

Activities(ActivityId, ActivityName)

Employees(EmployeeId, EmployeeName)

- Denormalize to Maintain History

Your company requires your system to maintain a customer's name from the time that you started a project for a customer. A customer can later change the name. How would you achieve this requirement?

Projects(ProjectId, ProjectName, CustomerId, **StartDate,**
CustomerName)

- Give the relation schema:

Projects(ProjectId, ProjectName, CustomerId)

ProjectDetails(ProjectId, ItemId, ActivityId, EmployeeId, WorkDate, TimeSpent) Customers(CustomerId, CustomerName)

Activities(ActivityId, ActivityName) Employees(EmployeeId, EmployeeName)

- Denormalize for Performance

You have performance problems with a report that calculates total time spent on a project. Because you are tracking multiple projects, you run this report multiple times a day. How would you improve report performance?

Projects(ProjectId, ProjectName, CustomerId, **StartDate,**
CustomerName, TotalTimeSpent)

Q & A

